Lab Assignment 6

Object-Oriented Programming

Branch

B.E., 2nd Year – ENC

Submitted By:
MRIDUL SHARMA

Roll No. 1024150204

Batch: 2O23

Q1 - Implement a class Library that contains an array of Book objects. The Book class should have attributes like *title, author, and ISBN*. The Library class should provide the following methods:

(a) bool addNewBook(title, author, ISBN)

- **Add a new book to the list**
- **Return type should be Boolean (true/false)**
- **Use pass-by-reference for arguments**
- **The function arguments and class attributes should have the same names •**

Use the scope resolution operator to access class data members (b) bool

removeBooks(ISBN)

- **Remove a book using ISBN**
- **Return Boolean (true/false)**
- **Use pass-by-reference**
- **Define this function outside the class using scope resolution operator**

(c) displayDetails()

- **Display all books in the library**

Write a main function to:

- **Add at least 5 books**
- **Remove 1 book**
- **Display all books Code:**

WRITTEN CODE

```
#include <iostream>

using namespace

std;  class Book {

public:

 string title, author, ISBN;

};  class

Library {

 Book books[10];
 int count;
public:
```

```
 Library() { count = 0; }  bool addNewBook(string &title, string
&author, string &ISBN) {  if (count >= 10) return false;
books[count].title = title;  books[count].author = author;
books[count].ISBN = ISBN;  count++;  return true;

 }
bool removeBooks(string &ISBN);
void displayDetails() {  for (int i =
0; i < count; i++) {  cout <<
books[i].title << " "  <<
books[i].author << " "
 << books[i].ISBN << endl;
 }
 }
};

bool Library::removeBooks(string &ISBN) {
for (int i = 0; i < count; i++) {  if
(books[i].ISBN == ISBN) {  for
(int j = i; j < count - 1; j++) {
books[j] = books[j + 1];
 }  count--;
return true;
 }
 }
return false;
}  int main()
{  Library
lib;  string t,
a, i;
 t="A"; a="AA"; i="1"; lib.addNewBook(t,a,i);
 t="B"; a="BB"; i="2"; lib.addNewBook(t,a,i);
```

```
t="C"; a="CC"; i="3"; lib.addNewBook(t,a,i);
t="D"; a="DD"; i="4"; lib.addNewBook(t,a,i);
t="E"; a="EE"; i="5"; lib.addNewBook(t,a,i);  string
rem = "3";  lib.removeBooks(rem);
lib.displayDetails();
}
```

```cpp
#include <iostream>
using namespace std;
class Book {
public:
    string title, author, ISBN;
};
class Library {
    Book books[10];
    int count;
public:
    Library() { count = 0; }
    bool addNewBook(string &title, string &author, string &ISBN) {
        if (count >= 10) return false;
        books[count].title = title;
        books[count].author = author;
        books[count].ISBN = ISBN;
        count++;
        return true;
    }
    bool removeBooks(string &ISBN);
    void displayDetails() {
        for (int i = 0; i < count; i++) {
            cout << books[i].title << " "
                 << books[i].author << " "
                 << books[i].ISBN << endl;
        }
    }
};

bool Library::removeBooks(string &ISBN) {
    for (int i = 0; i < count; i++) {
        if (books[i].ISBN == ISBN) {
            for (int j = i; j < count - 1; j++) {
                books[j] = books[j + 1];
            }
            count--;
            return true;
        }
    }
    return false;
}
int main() {
    Library lib;
    string t, a, i;
    t="A"; a="AA"; i="1"; lib.addNewBook(t,a,i);
    t="B"; a="BB"; i="2"; lib.addNewBook(t,a,i);
    t="C"; a="CC"; i="3"; lib.addNewBook(t,a,i);
    t="D"; a="DD"; i="4"; lib.addNewBook(t,a,i);
    t="E"; a="EE"; i="5"; lib.addNewBook(t,a,i);
    string rem = "3";
    lib.removeBooks(rem);
    lib.displayDetails();
}
```

```
A AA 1
B BB 2
D DD 4
E EE 5


...Program finished with exit code 0
Press ENTER to exit console.
```

Q2 - Implement Question 1 using constructors with the following modifications: (a) Add:

- **Default constructor**
- **Parameterized constructor**
- **Copy constructor to initialize book details (*title, author, ISBN*)**

(b) Use the this pointer instead of scope resolution operator to access class data members (c) Create function bool removeBooks(ISBN) same as Question 1

(d) Create function displayDetails() same as Question 1

(e) Create array of objects using:

- **Initializer list**
- **Dynamic initialization**

WRITTEN CODE

```
#include <iostream>
using namespace std;
class Book {  public:
string title, author, ISBN;
// default
 Book() {}
// parameterized
 Book(string title, string author, string ISBN)
{  this->title = title;  this->author = author;
this->ISBN = ISBN;

 }
// copy
 Book(const Book &b) {
 this->title = b.title;  this-
 >author = b.author;  this->ISBN
 = b.ISBN;
}  void display() {  cout << title << " " << author << " "
<< ISBN << endl; }
```

```
};  int

main() {

// initializer list
 Book arr[2] = {

 Book("A", "AA", "1"),

Book("B", "BB", "2") };

for (int i = 0; i < 2; i++)

arr[i].display();

// dynamic array

 Book *b = new Book[2]{
Book("C", "CC", "3"),

Book("D", "DD", "4") };

 for (int i = 0; i < 2;

i++)   b[i].display();

delete[] b;

}
```

```cpp
#include <iostream>
using namespace std;
class Book {
public:
    string title, author, ISBN;
    // default
    Book() {}
    // parameterized
    Book(string title, string author, string ISBN) {
        this->title = title;
        this->author = author;
        this->ISBN = ISBN;
    }
    // copy
    Book(const Book &b) {
        this->title = b.title;
        this->author = b.author;
        this->ISBN = b.ISBN;
    }
    void display() {
        cout << title << " " << author << " " << ISBN << endl;
    }
};
int main() {
    // initializer list
    Book arr[2] = {
        Book("A", "AA", "1"),
        Book("B", "BB", "2")
    };
    for (int i = 0; i < 2; i++)
        arr[i].display();
    // dynamic array
    Book *b = new Book[2]{
        Book("C", "CC", "3"),
        Book("D", "DD", "4")
    };
    for (int i = 0; i < 2; i++)
        b[i].display();
    delete[] b;
}
```

```
A AA 1
B BB 2
C CC 3
D DD 4


...Program finished with exit code 0
Press ENTER to exit console.
```

Q3 - Create a class Account with the following attributes:

- **account number *(const long type)***
- **transaction ID *(long type)***
- **transaction type *(credit/debit — string)***
- **balance *(double type)***

The class should provide the following methods:

(a) long depositAmount(to, from, amount)

- **"to" and "from" are account numbers**
- **Return the transaction ID**

(b) long creditAmount(to, from, amount)

- **Similar to depositAmount**
- **Return the transaction ID**

(c) void displayDetails()

- **Display account number, balance, and transaction history •**

  Make this a const function

(d) In all functions, make all arguments const

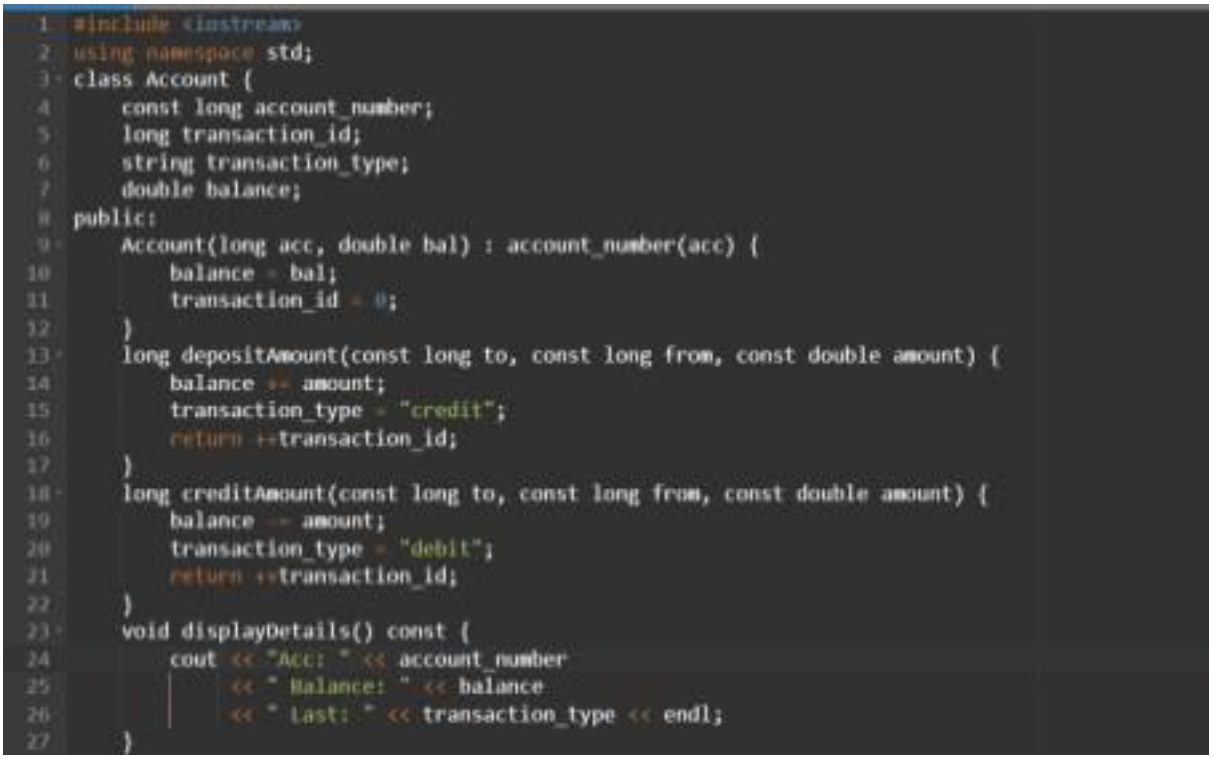
Write a main function to:

- **Create at least 5 Account objects**
- **Demonstrate credit and debit operations**

WRITTEN CODE

```
#include <iostream>  using

namespace std;  class

Account {  const long

account_number;  long

transaction_id;  string

transaction_type;  double

balance;  public:

Account(long acc, double bal) : account_number(acc) {

balance = bal;  transaction_id = 0;

 }

 long depositAmount(const long to, const long from, const double amount) {
balance += amount;
transaction_type = "credit";  return

++transaction_id;

 }  long creditAmount(const long to, const long from, const double

 amount) {  balance -= amount;  transaction_type = "debit";  return

 ++transaction_id;

}  void displayDetails() const {  cout

<< "Acc: " << account_number  <<

" Balance: " << balance
```

```
 << " Last: " << transaction_type << endl;
 }  };  int
main() {
Account a1(1,1000), a2(2,2000), a3(3,3000),
a4(4,4000), a5(5,5000);
a1.depositAmount(1,2,500);
a2.creditAmount(2,1,300);
a1.displayDetails();  a2.displayDetails();
a3.displayDetails();  a4.displayDetails();
a5.displayDetails();
}
```



```cpp
#include <iostream>
using namespace std;
class Account {
    const long account_number;
    long transaction_id;
    string transaction_type;
    double balance;
public:
    Account(long acc, double bal) : account_number(acc) {
        balance = bal;
        transaction_id = 0;
    }
    long depositAmount(const long to, const long from, const double amount) {
        balance += amount;
        transaction_type = "credit";
        return ++transaction_id;
    }
    long creditAmount(const long to, const long from, const double amount) {
        balance -= amount;
        transaction_type = "debit";
        return ++transaction_id;
    }
    void displayDetails() const {
        cout << "Acc: " << account_number
             << " Balance: " << balance
             << " Last: " << transaction_type << endl;
    }
};
int main() {
    Account a1(1,1000), a2(2,2000), a3(3,3000),
            a4(4,4000), a5(5,5000);
    a1.depositAmount(1,2,500);
    a2.creditAmount(2,1,300);
    a1.displayDetails();
    a2.displayDetails();
    a3.displayDetails();
    a4.displayDetails();
    a5.displayDetails();
}
```

```
Acc: 1 Balance: 1500 Last: credit
Acc: 2 Balance: 1700 Last: debit
Acc: 3 Balance: 3000 Last:
Acc: 4 Balance: 4000 Last:
Acc: 5 Balance: 5000 Last:


...Program finished with exit code 0
Press ENTER to exit console.
```

Q4 - Write a program to add data objects of two different classes using friend function.

WRITTEN CODE

```
#include <iostream>
using namespace std;
class B;
class A {  int
x;  public:
```

A() { x = 10; }

friend int sum(A, B);

}; class B { int y;

public: B() { y = 20;

} friend int sum(A,

B);

}; int sum(A a, B

b) { return a.x +

b.y; } int main()

{

A a; B b; cout << "Sum = " <<

sum(a, b); }

```cpp
#include <iostream>
using namespace std;
class B;
class A {
    int x;
public:
    A() { x = 10; }
    friend int sum(A, B);
};
class B {
    int y;
public:
    B() { y = 20; }
    friend int sum(A, B);
};
int sum(A a, B b) {
    return a.x + b.y;
}
int main() {
    A a;
    B b;
    cout << "Sum = " << sum(a, b);
}
```

```
Sum = 30

...Program finished with exit code 0
Press ENTER to exit console.
```

Q5

Define a class **Complex** with properties *(real and imaginary)* and implement:

(a) Parameterized constructor and copy constructor to initialize object values

(b) void display() to display the complex number

(c) void sum(Complex, Complex)

- Add two complex numbers
- Implement this as a **friend function**

Code:

```
#include <iostream>
using namespace std;
class Complex {  float
real, imag;
public:
 Complex(float r = 0, float i = 0)
{   real = r;  imag = i;
 }
 Complex(const Complex &c) {
real = c.real;
 imag = c.imag;
}  void display() {  cout << real << " + "
<< imag << "i\n"; }  friend Complex
sum(Complex, Complex);
};
Complex sum(Complex c1, Complex c2)
{ Complex temp;  temp.real =
c1.real + c2.real;  temp.imag =
c1.imag + c2.imag;   return
temp;
}  int
main() {
Complex
c1(2,3),
c2(4,5);
Complex
c3 =
sum(c1,
```

c2);

c3.display(

);

}

```cpp
#include <iostream>
using namespace std;
class Complex {
    float real, imag;
public:
    Complex(float r = 0, float i = 0) {
        real = r;
        imag = i;
    }
    Complex(const Complex &c) {
        real = c.real;
        imag = c.imag;
    }
    void display() {
        cout << real << " + " << imag << "i\n";
    }
    friend Complex sum(Complex, Complex);
};
Complex sum(Complex c1, Complex c2) {
    Complex temp;
    temp.real = c1.real + c2.real;
    temp.imag = c1.imag + c2.imag;
    return temp;
}
int main() {
    Complex c1(2,3), c2(4,5);
    Complex c3 = sum(c1, c2);
    c3.display();
}
```

```
6 + 8i


...Program finished with exit code 0
Press ENTER to exit console.
```